

# Machine Learning — Part 3

## Study Notes: Classification Algorithms — Full Mathematical Derivations & Worked Examples

---

These notes rebuild every mathematical derivation from the lecture on classification algorithms — **Logistic Regression, KNN, Naive Bayes, Decision Trees, and SVM** — plus model-evaluation metrics (Confusion Matrix, Accuracy, Precision, Recall, F1) and K-Fold Cross Validation. Every formula here is worked through with a small, original numeric dataset built specifically for this document, and every number was computed with Python so the arithmetic is verified and exact — not just described in words.

**How this differs from a transcript:** Rather than reproducing the video's own dataset, this guide uses freshly constructed small datasets for each algorithm so every step — every entropy value, every probability, every distance — is a real, checkable calculation.

# 1. What Is Classification?

---

Classification predicts a **category or class label** for a given input, as opposed to regression, which predicts a continuous number. Typical questions: Is this email spam or not? Will this patient develop heart disease or not? Which of 3 species is this flower?

- **Binary classification:** exactly two classes (0/1, Yes/No, Spam/Not-Spam).
- **Multi-class classification:** more than two classes (e.g. Setosa / Versicolor / Virginica).

## Classification Model Workflow

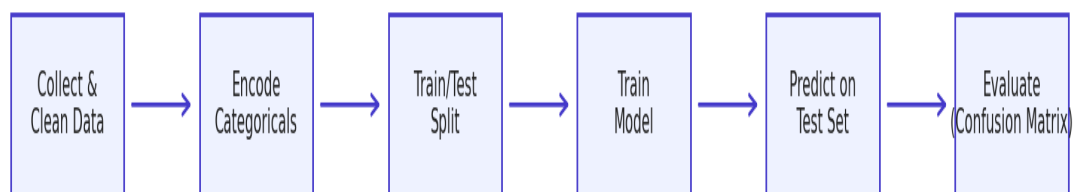


Fig 1. The general workflow for building and evaluating any classification model.

This guide covers five classifiers in the order they are usually taught: Logistic Regression → K-Nearest Neighbours (KNN) → Naive Bayes → Decision Trees → Support Vector Machines (SVM) — followed by how to evaluate any of them using a confusion matrix.

## 2. Logistic Regression

---

### 2.1 Why Not Just Use Linear Regression?

A classification target only takes values like 0 or 1. Fitting an ordinary straight line ( $h(x) = \theta_0 + \theta_1 x$ ) to 0/1 data creates two problems:

- **Outliers distort the line** — a single extreme data point can drag the whole straight line, shifting where it crosses the halfway mark and flipping predictions for other points.
- **Predictions are unbounded** — a straight line can output values below 0 or above 1, which makes no sense as a probability of belonging to a class.

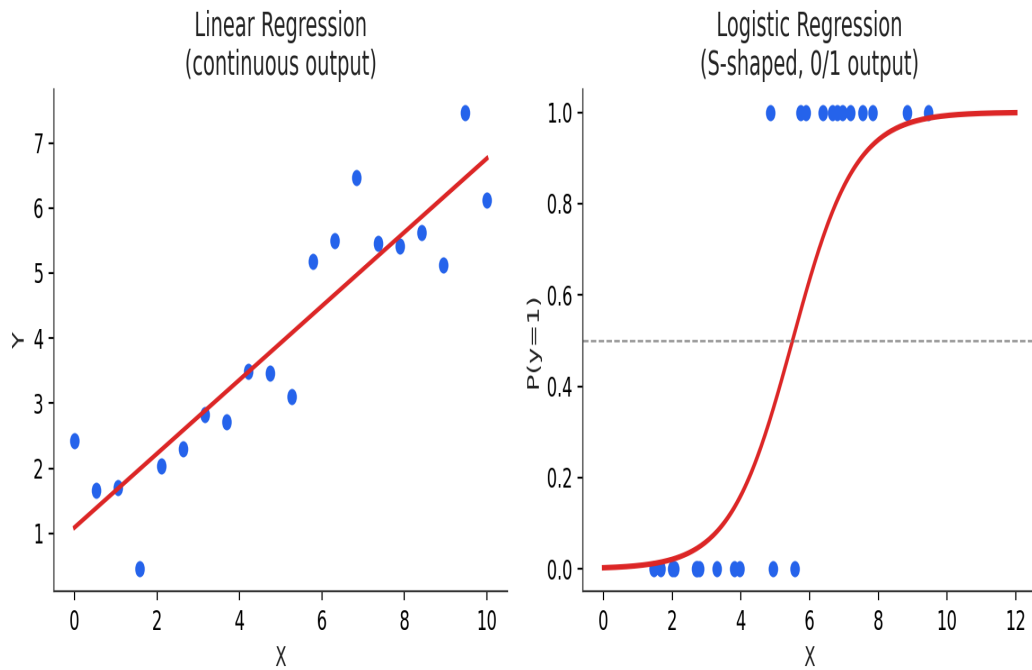


Fig 2. Linear regression fits a straight line for continuous output; logistic regression fits an S-shaped curve bounded between 0 and 1 for 0/1 output.

## 2.2 The Sigmoid (Logistic) Function

Logistic Regression starts from the same linear formula as Linear Regression, then "squashes" it into an S-shaped curve using the **sigmoid function**, so the output always lands between 0 and 1 and can be read as a probability:

$$z = \theta_0 + \theta_1 x$$

$$h_{\theta}(x) = g(z) = 1 / (1 + e^{-z})$$

Here  $e \approx 2.71828$  is Euler's number, the base of the natural logarithm — its role is purely to produce a smooth, continuously curving S-shape rather than a jagged line.

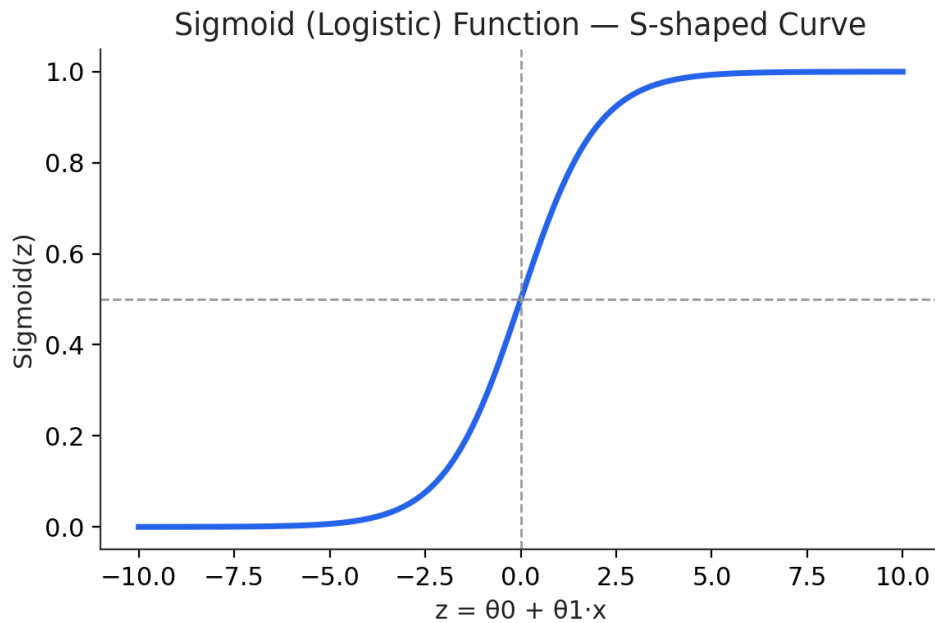


Fig 3. The sigmoid function — any real-valued input  $z$  is compressed into the range  $(0, 1)$ .

## 2.3 Worked Example — Predicting Pass/Fail from Study Hours

Suppose a simple model has learned  $\theta_0 = -4$  and  $\theta_1 = 0.08$  for predicting whether a student passes (1) or fails (0) based on hours studied. Plugging in different values of  $x$  gives:

| Hours studied ( $x$ ) | $z = -4 + 0.08x$ | Sigmoid( $z$ ) = P(pass)           |
|-----------------------|------------------|------------------------------------|
| 20                    | -2.400           | 0.0832 (8.3%)                      |
| 40                    | -0.800           | 0.3100 (31.0%)                     |
| 50                    | 0.000            | 0.5000 (50.0%) ← decision boundary |
| 60                    | 0.800            | 0.6900 (69.0%)                     |
| 80                    | 2.400            | 0.9168 (91.7%)                     |

Whenever the predicted probability is  $\geq 0.5$ , the model classifies the point as 1 (pass); below 0.5, as 0 (fail). At exactly  $x = 50$  hours the probability is exactly 50% — this is the **decision boundary**.

## 2.4 Log Loss (Binary Cross-Entropy) — the Cost Function

Linear Regression minimizes the Mean Squared Error between predicted and actual values. Logistic Regression instead minimizes **Log Loss** (also called Binary Cross-Entropy), because it penalizes confidently wrong predictions far more heavily than a simple squared difference would:

$$J(\theta) = -(1/m) \cdot \sum_{i=1}^m [y_i \cdot \log(\hat{y}_i) + (1-y_i) \cdot \log(1-\hat{y}_i)]$$

Where  $y_i$  is the true label (0 or 1) and  $\hat{y}_i$  is the predicted probability. Worked example for four individual points:

| Actual $y$ | Predicted probability $\hat{y}$ | Loss for this point                  |
|------------|---------------------------------|--------------------------------------|
| 1          | 0.90                            | 0.1054 (small — confident & correct) |

|   |      |                                      |
|---|------|--------------------------------------|
| 1 | 0.60 | 0.5108 (larger — less confident)     |
| 0 | 0.20 | 0.2231 (small — confident & correct) |
| 0 | 0.55 | 0.7985 (large — confidently wrong)   |

Average log loss across these four points = **0.4095**. Notice the last row: predicting 0.55 when the true label is 0 produces a much bigger penalty than being simply "a bit off" — log loss punishes over-confident wrong answers steeply, which is exactly the behaviour needed for a good classifier.

Just like Linear Regression, the values of  $\theta_0$  and  $\theta_1$  that minimize this cost function are found using **Gradient Descent** — repeatedly nudging  $\theta_0$  and  $\theta_1$  in the direction that reduces the log loss, controlled by a learning rate  $\alpha$ , until the cost stops improving.

### 3. Evaluating a Classifier: The Confusion Matrix

For classification, accuracy alone can be misleading, so results are first organized into a **confusion matrix**, comparing predicted labels against actual labels:

- **True Positive (TP)** — predicted positive, actually positive.
- **False Positive (FP)** — predicted positive, actually negative (a "Type I error").
- **False Negative (FN)** — predicted negative, actually positive (a "Type II error").
- **True Negative (TN)** — predicted negative, actually negative.

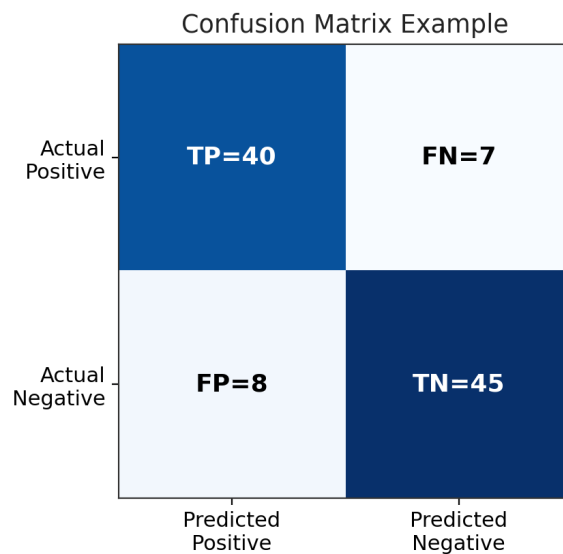


Fig 4. A worked confusion matrix: TP=40, FN=7, FP=8, TN=45 (100 total predictions).

#### 3.1 The Four Key Metrics

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{F1 Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Using the numbers from Fig 4 (TP=40, FP=8, FN=7, TN=45):

| Metric    | Formula substitution                                  | Result         |
|-----------|-------------------------------------------------------|----------------|
| Accuracy  | $(40+45)/(40+45+8+7)$                                 | 0.8500 (85.0%) |
| Precision | $40/(40+8)$                                           | 0.8333 (83.3%) |
| Recall    | $40/(40+7)$                                           | 0.8511 (85.1%) |
| F1 Score  | $2 \times (0.8333 \times 0.8511) / (0.8333 + 0.8511)$ | 0.8421 (84.2%) |

#### 3.2 When to Use Which Metric

| Metric    | Use when...                                                               | Example                                                       |
|-----------|---------------------------------------------------------------------------|---------------------------------------------------------------|
| Accuracy  | Classes are roughly balanced and every correct prediction matters equally | General classification task                                   |
| Precision | False Positives are costly                                                | Spam detection — don't want to bury real emails as spam       |
| Recall    | False Negatives are costly                                                | Disease detection — don't want to miss an actual sick patient |
| F1 Score  | You need a single balanced number combining precision and recall          | Imbalanced classes, e.g. survival prediction                  |

Precision and Recall represent a trade-off: pushing one up by adjusting the decision threshold usually pulls the other down. F1 is the **harmonic mean** of the two, so it only stays high when both are reasonably high — a large gap between them drags F1 down more than a simple average would.

## 4. K-Nearest Neighbours (KNN)

KNN makes no assumption about the shape of the data. To classify a new point, it simply looks at the **K closest existing points** (by distance) and takes a majority vote of their labels.

### 4.1 Algorithm Steps

- 1. Choose a value of K (must be odd for binary classification, to avoid a tie).
- 2. Compute the distance from the new point to every point in the training data — usually **Euclidean distance**.
- 3. Sort all distances from smallest to largest.
- 4. Take the K nearest neighbours and count the majority class among them.
- 5. Assign that majority class as the prediction (for regression tasks, take the mean instead of the majority vote).

$$\text{distance(A, B)} = \sqrt{[(x_{1A} - x_{1B})^2 + (x_{2A} - x_{2B})^2 + \dots]}$$

### 4.2 Worked Example

A small dataset of (hours studied, practice tests taken) → Pass(1)/Fail(0):

| Hours studied | Practice tests | Result |
|---------------|----------------|--------|
| 1             | 1              | Fail   |
| 2             | 1              | Fail   |
| 2             | 3              | Fail   |
| 4             | 2              | Pass   |
| 5             | 4              | Pass   |
| 6             | 5              | Pass   |
| 3             | 1              | Fail   |
| 5             | 3              | Pass   |

New student: 4 hours studied, 3 practice tests. Distance to every existing point (sorted, using K = 3):

```
distance((4,3),(4,2)) = sqrt((4-4)^2+(3-2)^2) = 1.000 -> label: Pass
distance((4,3),(5,3)) = sqrt((4-5)^2+(3-3)^2) = 1.000 -> label: Pass
distance((4,3),(5,4)) = sqrt((4-5)^2+(3-4)^2) = 1.414 -> label: Pass
distance((4,3),(2,3)) = sqrt((4-2)^2+(3-3)^2) = 2.000 -> label: Fail
distance((4,3),(3,1)) = sqrt((4-3)^2+(3-1)^2) = 2.236 -> label: Fail
distance((4,3),(2,1)) = sqrt((4-2)^2+(3-1)^2) = 2.828 -> label: Fail
distance((4,3),(6,5)) = sqrt((4-6)^2+(3-5)^2) = 2.828 -> label: Pass
distance((4,3),(1,1)) = sqrt((4-1)^2+(3-1)^2) = 3.606 -> label: Fail
```

```
K=3 nearest neighbours: (4,2)=Pass, (5,3)=Pass, (5,4)=Pass
Majority vote: Pass (3 out of 3) -> PREDICTION = Pass
```

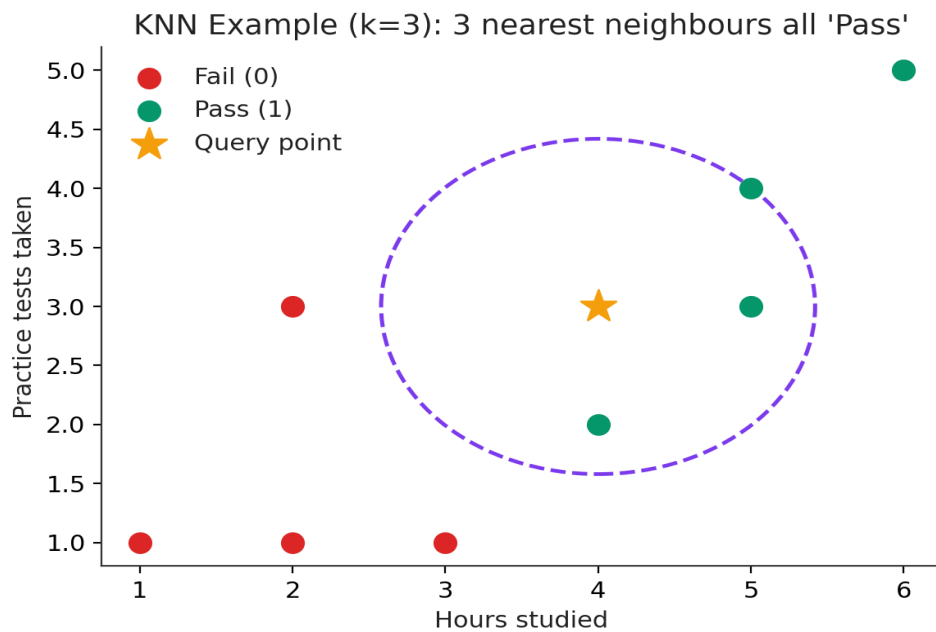


Fig 5. The query point (star) and its 3 nearest neighbours (inside the dashed circle), all labelled Pass, so the majority-vote prediction is Pass.

### 4.3 Choosing K

- K should be an **odd number** for binary classification, so a majority vote never ties.
- K is a **hyperparameter** — its best value is found experimentally, typically via **K-Fold Cross Validation** (train and evaluate several models with different K values, e.g. 1, 3, 5, 7... and keep whichever K gives the best average validation accuracy/precision).

Because KNN is entirely distance-based, **feature scaling (e.g. StandardScaler) is essential** before using it — a feature with naturally larger numeric values (like income in dollars vs. age in years) would otherwise dominate the distance calculation purely because of its scale, not its real importance.

## 5. Naive Bayes

### 5.1 Probability Foundations: Independent vs. Dependent Events

**Independent events** — one event's outcome does not affect the other's probability. Example: rolling a die twice — getting a 6 on the first roll doesn't change the 1/6 probability of any number on the second roll.

$$P(A \text{ and } B) = P(A) \times P(B) \text{ [independent events]}$$

**Dependent events** — one event's outcome changes the probability of the next. Example: drawing two cards from a 52-card deck without replacement — after removing one red card, only 25 of the remaining 51 cards are red, so  $P(\text{red on 2nd draw}) = 25/51$ , not  $26/52$ .

$$P(A \text{ and } B) = P(A) \times P(B | A) \text{ [dependent events — "conditional probability"]}$$

### 5.2 Deriving Bayes' Theorem

Because  $P(A \text{ and } B)$  is symmetric — it doesn't matter which event is labelled "first" — the two ways of expanding it must be equal:

$$P(A) \times P(B|A) = P(B) \times P(A|B)$$

Rearranging this single line for  $P(A|B)$  gives Bayes' Theorem:

$$P(A | B) = [ P(A) \times P(B | A) ] / P(B)$$

### 5.3 Applying Bayes' Theorem to Classification

For a classification problem with independent features  $x_1, x_2, \dots, x_n$  predicting output  $y$ , Bayes' Theorem becomes:

$$P(y | x_1, \dots, x_n) = [ P(y) \times P(x_1|y) \times P(x_2|y) \times \dots \times P(x_n|y) ] / P(x_1, \dots, x_n)$$

This formula is computed once per class (e.g. once assuming "Yes", once assuming "No"), and whichever class gives the higher result is the prediction. Because the denominator is the same for every class, it's often skipped — only the relative sizes of the numerators matter for deciding the winner.

### 5.4 Worked Example — Play Tennis Given Outlook = Sunny

A small 14-row training set records whether tennis is played, broken down by weather outlook: overall 9 rows say "Play", 5 say "Don't Play". Within the rows where Outlook = Sunny specifically, 2 say "Play" and 3 say "Don't Play" (5 sunny rows total, out of 14 overall).

```
P(Play) = 9/14 = 0.643          P(Don't Play) = 5/14 = 0.357
P(Sunny | Play) = 2/9 = 0.222    P(Sunny | Don't Play) = 3/5 = 0.600

Unnormalized P(Play | Sunny)      = P(Play) x P(Sunny|Play)      = 0.643 x 0.222
= 0.143
Unnormalized P(Don't Play | Sunny) = P(Don't Play) x P(Sunny|Don't Play) = 0.357
x 0.600 = 0.214

Normalizing (dividing both by their sum, 0.357):
P(Play | Sunny)      = 0.143 / 0.357 = 0.400 (40%)
```

$$P(\text{Don't Play} \mid \text{Sunny}) = 0.214 / 0.357 = 0.600 \quad (60\%)$$

Since  $60\% > 40\%$   $\rightarrow$  PREDICTION = Don't Play

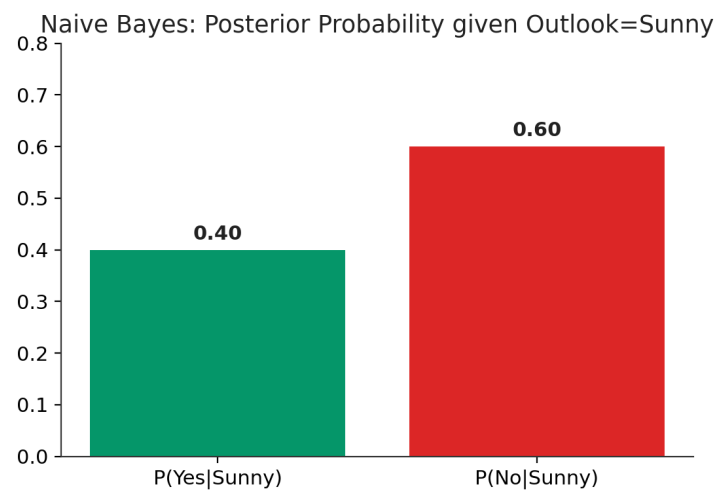


Fig 6. Even though more days overall were "Play" days, conditioning on Outlook=Sunny flips the prediction to "Don't Play".

This is called **Naive** Bayes because it assumes every feature is conditionally independent of every other feature given the class — rarely perfectly true in real data, but the approximation works surprisingly well in practice. When features are continuous (like age or income), a common variant called **Gaussian Naive Bayes** assumes each feature follows a normal (bell-curve) distribution within each class.

## 6. Decision Trees

### 6.1 Structure

A decision tree mirrors a chain of if/else questions. Each internal node asks a yes/no or multi-way question about one feature; each branch is an answer; each leaf is a final prediction.

- **Root node** — the very first, most informative question.
- **Branches** — the possible answers, leading to further questions or to leaves.
- **Leaf nodes** — terminal nodes giving the final predicted class.

### 6.2 Entropy — Measuring Impurity

**Entropy** measures how mixed (impure) a set of labels is. It is 0 when a group is perfectly pure (all one class) and reaches its maximum when the group is split as evenly as possible between classes:

$$\text{Entropy}(S) = -p_+ \cdot \log_2(p_+) - p_- \cdot \log_2(p_-)$$

Where  $p_+$  and  $p_-$  are the proportions of positive and negative examples in the set.

### 6.3 Information Gain — Choosing the Best Feature to Split On

To decide which feature becomes the root node (or any subsequent node), compute how much entropy is reduced by splitting the data on that feature — this reduction is the **Information Gain**:

$$\text{IG}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{values}(A)} \left[ \frac{|S_v|}{|S|} \right] \times \text{Entropy}(S_v)$$

Where  $S_v$  is the subset of  $S$  where feature  $A$  takes value  $v$ , and  $|S_v|/|S|$  is the proportion of the data that falls into that branch. Whichever feature produces the **highest** Information Gain is chosen for that node.

### 6.4 Worked Example — Choosing a Root Node

Using the same 14-row Play Tennis dataset from Section 5.4 (9 Play, 5 Don't Play), with four candidate features (Outlook, Temperature, Humidity, Wind):

```
Entropy(S) = -(9/14)*log2(9/14) - (5/14)*log2(5/14) = 0.9403

-- OUTLOOK -- Sunny(2Y,3N) Overcast(4Y,0N) Rain(3Y,2N)
Entropy(Sunny)=0.9710 Entropy(Overcast)=0.0000 Entropy(Rain)=0.9710
IG(Outlook) = 0.9403 - [(5/14)*0.9710 + (4/14)*0 + (5/14)*0.9710] = 0.2467

-- HUMIDITY -- High(3Y,4N) Normal(6Y,1N)
Entropy(High)=0.9852 Entropy(Normal)=0.5917
IG(Humidity) = 0.9403 - [(7/14)*0.9852 + (7/14)*0.5917] = 0.1518

-- WIND -- Weak(6Y,2N) Strong(3Y,3N)
Entropy(Weak)=0.8113 Entropy(Strong)=1.0000
IG(Wind) = 0.9403 - [(8/14)*0.8113 + (6/14)*1.0000] = 0.0481

-- TEMPERATURE -- Hot(2Y,2N) Mild(4Y,2N) Cool(3Y,1N)
Entropy(Hot)=1.0000 Entropy(Mild)=0.9183 Entropy(Cool)=0.8113
IG(Temperature) = 0.9403 - [(4/14)*1.0 + (6/14)*0.9183 + (4/14)*0.8113] = 0.0292
```

RESULT:  $IG(\text{Outlook})=0.2467$  is the HIGHEST -> Outlook becomes the ROOT NODE

Decision Tree: Root Node chosen by highest Information Gain

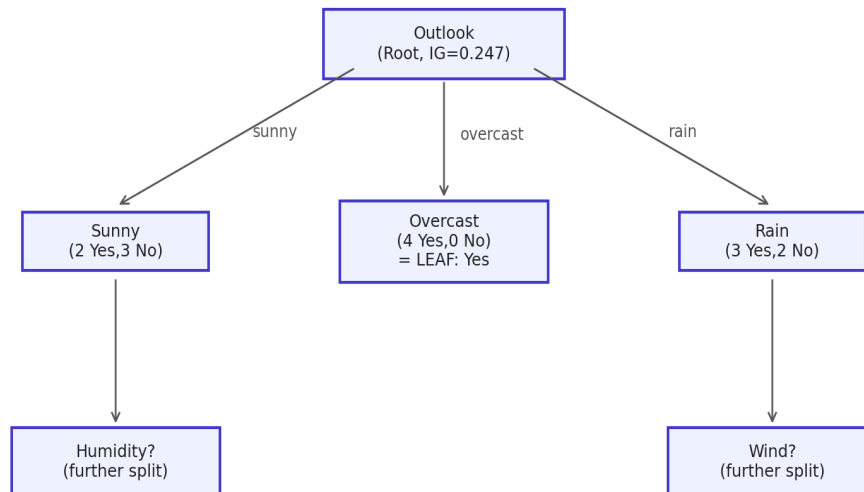


Fig 7. Outlook is chosen as the root because it has the highest Information Gain. The Overcast branch is already 100% pure (4 Yes, 0 No) so it becomes a leaf immediately; the Sunny and Rain branches still need further splitting.

Once the root is fixed, the same entropy/information-gain calculation is **repeated recursively** inside each impure branch (e.g. splitting the Sunny rows further by Humidity or Temperature), building the tree deeper until every branch is either pure or a stopping condition is met.

## 6.5 Stopping Conditions

- All data at a node is pure (entropy = 0) — that node becomes a leaf.
- A maximum tree **depth** is reached (a hyperparameter set in advance, to prevent the tree from growing so deep it overfits the training data).
- Too few samples remain at a node to justify another split.

Decision Trees are the building block for **Random Forest** (an ensemble of many decision trees voting together) — covered as the next step after this in a full curriculum.

## 7. Support Vector Machines (SVM)

---

### 7.1 Core Idea

Given two groups of points, there are infinitely many lines that could separate them. SVM picks the **one specific line that maximizes the distance (margin) to the closest point of each class** — this is the most robust choice, since it leaves the largest possible buffer against misclassifying new, slightly-off points.

- **Hyperplane** — the separating line (in 2D) or plane (in 3D+) itself.
- **Margin** — the space between the hyperplane and the nearest point of either class.
- **Support vectors** — the specific data points closest to the hyperplane that "support"/define the margin; removing any other point wouldn't change the boundary, but moving a support vector would.

### 7.2 The Decision Formula

$$\mathbf{w} \cdot \mathbf{x} + b = 0 \text{ (the hyperplane itself)}$$

Where  $\mathbf{w}$  is a weight vector (like the slope in Linear/Logistic Regression) and  $b$  is the bias (like the intercept). For a new point  $\mathbf{x}$ :

- If  $\mathbf{w} \cdot \mathbf{x} + b > 0 \rightarrow$  predict **Class +1**
- If  $\mathbf{w} \cdot \mathbf{x} + b < 0 \rightarrow$  predict **Class -1**

### 7.3 Worked Example

Six 2-D points with  $\mathbf{w} = (1, 1)$  and  $b = -4$ , giving the hyperplane  $x_1 + x_2 = 4$ :

|             |                                                |                 |             |         |
|-------------|------------------------------------------------|-----------------|-------------|---------|
| point=(1,1) | $\mathbf{w} \cdot \mathbf{x} + b = 1+1-4 = -2$ | -> predicted -1 | (actual -1) | correct |
| point=(1,2) | $\mathbf{w} \cdot \mathbf{x} + b = 1+2-4 = -1$ | -> predicted -1 | (actual -1) | correct |
| point=(2,1) | $\mathbf{w} \cdot \mathbf{x} + b = 2+1-4 = -1$ | -> predicted -1 | (actual -1) | correct |
| point=(3,4) | $\mathbf{w} \cdot \mathbf{x} + b = 3+4-4 = 3$  | -> predicted +1 | (actual +1) | correct |
| point=(4,3) | $\mathbf{w} \cdot \mathbf{x} + b = 4+3-4 = 3$  | -> predicted +1 | (actual +1) | correct |
| point=(4,4) | $\mathbf{w} \cdot \mathbf{x} + b = 4+4-4 = 4$  | -> predicted +1 | (actual +1) | correct |

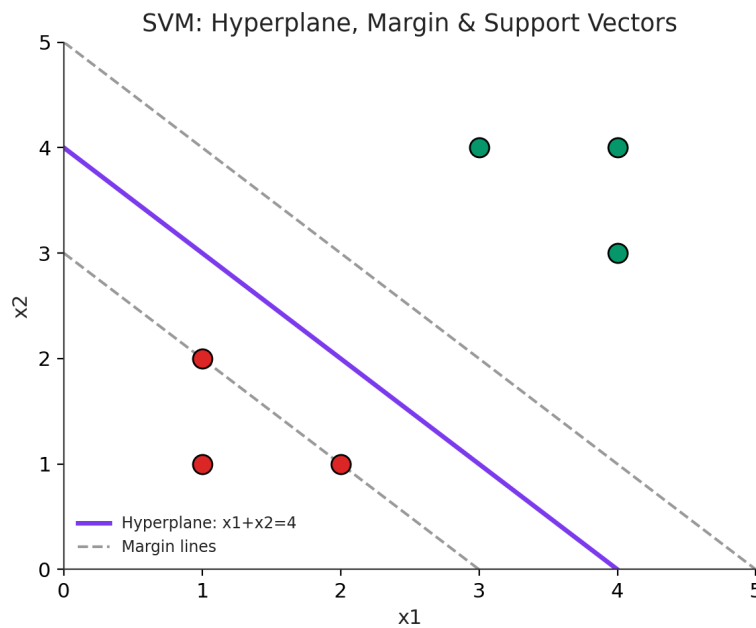


Fig 8. The solid hyperplane  $x_1+x_2=4$  separates the two classes; the dashed lines mark the margin boundaries on each side.

## 7.4 The Kernel Trick (Non-Linearly-Separable Data)

Sometimes no straight line can separate the classes in the original number of dimensions — e.g. one class forms a ring around the other. SVM handles this with a **kernel**: a function that projects the data into a higher number of dimensions where a flat hyperplane *can* separate the classes, then maps that separation back down. Common kernels include:

| Kernel                      | When it's used                                                        |
|-----------------------------|-----------------------------------------------------------------------|
| Linear                      | Data is already (roughly) linearly separable                          |
| RBF (Radial Basis Function) | Most widely used default; handles complex, non-linear boundaries well |
| Polynomial                  | Curved boundaries with a specific polynomial degree                   |
| Sigmoid                     | Behaves similarly to a neural network activation                      |

The mathematics behind exactly how a kernel computes similarity in higher dimensions without explicitly transforming every point (the "kernel trick") is genuinely advanced linear algebra — in practice, picking `kernel='rbf'` in scikit-learn (the library default) is enough for the large majority of real projects; the point to remember for interviews is *why* a kernel is needed, not how to derive it by hand.

## 8. Common scikit-learn Implementation Pattern

---

All five classifiers in this guide are trained and evaluated with the same shape of code — only the imported class changes:

```
# Shared preprocessing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42)

scaler = StandardScaler() # needed for KNN / SVM
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# --- swap only this block between algorithms ---
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
# from sklearn.neighbors import KNeighborsClassifier
# model = KNeighborsClassifier(n_neighbors=5)
# from sklearn.naive_bayes import GaussianNB
# model = GaussianNB()
# from sklearn.tree import DecisionTreeClassifier
# model = DecisionTreeClassifier(random_state=42)
# from sklearn.svm import SVC
# model = SVC(kernel='rbf')
# -----

model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

The typical workflow: try several models on the same train/test split, compare accuracy and F1 score, and keep whichever performs best — different algorithms suit different datasets, so there is no single "always best" classifier.

### 8.1 K-Fold Cross Validation

Instead of relying on a single train/test split (which could be lucky or unlucky), K-Fold Cross Validation splits the data into K equal folds, trains K separate times (each time holding out a different fold for testing), and averages the resulting scores — giving a far more reliable estimate of real-world performance, and a principled way to choose hyperparameters like K in KNN.

## 9. Quick Reference: All Formulas

| Concept                         | Formula                                                             |
|---------------------------------|---------------------------------------------------------------------|
| Sigmoid function                | $g(z) = 1 / (1 + e^{-z})$                                           |
| Logistic hypothesis             | $h_{\theta}(x) = g(\theta_0 + \theta_1 x)$                          |
| Log Loss / Binary Cross-Entropy | $-(1/m) \sum [y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$ |
| Accuracy                        | $(TP+TN) / (TP+TN+FP+FN)$                                           |
| Precision                       | $TP / (TP+FP)$                                                      |
| Recall                          | $TP / (TP+FN)$                                                      |
| F1 Score                        | $2 \cdot (Precision \cdot Recall) / (Precision + Recall)$           |
| Euclidean distance (KNN)        | $\sqrt{[\sum (x_{iA} - x_{iB})^2]}$                                 |
| Bayes' Theorem                  | $P(A B) = P(A) \cdot P(B A) / P(B)$                                 |
| Entropy                         | $-p_+ \log_2(p_+) - p_- \log_2(p_-)$                                |
| Information Gain                | $Entropy(S) - \sum ( S_v / S ) \cdot Entropy(S_v)$                  |
| SVM hyperplane                  | $w \cdot x + b = 0$                                                 |

## 10. Summary Checklist

- I can explain why Linear Regression is unsuitable for 0/1 classification targets.
- I can compute the sigmoid function and read its output as a probability.
- I understand Log Loss and why it penalizes confident wrong predictions heavily.
- I can build a confusion matrix and compute Accuracy, Precision, Recall, and F1 by hand.
- I know when to prioritize Precision (spam) vs. Recall (disease detection).
- I can compute Euclidean distance and perform a KNN majority vote for a chosen K.
- I understand why K should be odd and why feature scaling matters for KNN.
- I can derive Bayes' Theorem from the dependent-events probability formula.
- I can compute Entropy and Information Gain to choose a Decision Tree's root node.
- I understand what a hyperplane, margin, and support vector are in SVM.
- I know why a kernel (e.g. RBF) is needed for non-linearly-separable data.
- I understand why K-Fold Cross Validation gives a more reliable score than one train/test split.

Next in the series: **Ensemble techniques** (Random Forest, Bagging, Boosting) and **Unsupervised Learning**.